

Documentación de la API REST en Local

Proyecto Node.js + Express + MongoDB

Índice

1. Introducción	3
2. Tecnologías Utilizadas	3
3. Estructura del Proyecto	4
4. Modelos de Datos	4
4.1. Modelo Usuario	4
4.2. Modelo Grupo	5
5. Servidor (server.js)	5
5.1. Descripción General	5
5.2. Código del Servidor	6
6. Endpoints de la API	10
6.1. Usuarios	10
7. Pruebas con Postman	10
7.1. GET de Usuarios	11
7.2. POST de Usuarios	12
7.3. DELETE de Usuarios	12
7.4. UPDATE de Usuarios	13
8. Docker Compose	15
9. Creación de la imagen de la API	18
9.1. Subir la imagen a Docker Hub	18
10. Workflow de GitHub	19
10.1. Creación del Workflow	19

1. Introducción

[Volver al índice](#)

Esta documentación describe el funcionamiento de una API REST creada con **Node.js**, **Express** y **MongoDB**, utilizando **Mongoose**.

La API proporciona operaciones CRUD para dos entidades principales: **Usuarios** y **Grupos**.

El servidor se ejecuta en un entorno local.

2. Tecnologías Utilizadas

[Volver al índice](#)

- Node.js — Entorno de ejecución para JavaScript.
- Express — Framework para crear el servidor HTTP.
- MongoDB — Base de datos NoSQL documental.
- Mongoose — ODM para manejar modelos y esquemas.
- Postman — Herramienta para probar los endpoints.

Instalación realizada:

```
pm init -y
npm install express mongoose
```

3. Estructura del Proyecto

[Volver al índice](#)

```
ombreProyecto/  
  
    models/  
        usuario.js  
        grupo.js  
    node_modules/  
    server.js  
    package.json
```

4. Modelos de Datos

[Volver al índice](#)

En nuestro proyecto tendremos dos modelos: uno para usuarios y otro para grupos. A continuación se muestra la estructura de cada uno.

4.1. Modelo Usuario

Archivo usuario.js:

```
import mongoose from "mongoose";  
  
const usuarioSchema = new mongoose.Schema({  
  nombre: { type: String, required: true },  
  email: { type: String, required: true, unique: true },  
  edad: { type: Number }  
}, {  
  versionKey: false  
});  
  
const Usuario = mongoose.model("Usuario", usuarioSchema);  
  
export default Usuario;
```

Campos del modelo Usuario:

- nombre — String (obligatorio)
- email — String, único (obligatorio)
- edad — Number (opcional)

4.2. Modelo Grupo

Archivo `grupo.js`:

```
import mongoose from "mongoose";

const grupoSchema = new mongoose.Schema({
  nombreGrupo: { type: String, required: true },
  cantidadAlumnos: { type: Number, required: true }
}, {
  versionKey: false
});

const Grupo = mongoose.model("Grupo", grupoSchema);

export default Grupo;
```

Campos del modelo Grupo:

- nombreGrupo — String (obligatorio)
- cantidadAlumnos — Number (obligatorio)

5. Servidor (server.js)

[Volver al índice](#)

5.1. Descripción General

El archivo `server.js` se encarga de:

- Conectar con MongoDB.
- Insertar datos iniciales si las colecciones están vacías.

- Definir endpoints CRUD para Usuarios y Grupos.
- Iniciar el servidor en `http://localhost:3000`.

5.2. Código del Servidor

```
import express from "express";
import mongoose from "mongoose";
import Usuario from "../models/usuario.js";
import Grupo from "../models/grupo.js";

const app = express();
app.use(express.json());

async function iniciarServidor() {
  const uri = process.env.MONGO_URI || "mongodb://localhost:27017/miBaseDeDatos";

  try {
    await mongoose.connect(uri);
    console.log("Mongoose conectado a MongoDB local");

    // Crear usuarios iniciales
    const usuariosExistentes = await Usuario.countDocuments();
    if (usuariosExistentes === 0) {
      await Usuario.insertMany([
        { nombre: "Ana", email: "ana@email.com", edad: 25 },
        { nombre: "Luis", email: "luis@email.com", edad: 30 }
      ]);
      console.log("Usuarios de ejemplo creados");
    }

    // Crear grupos iniciales
    const gruposExistentes = await Grupo.countDocuments();
    if (gruposExistentes === 0) {
      await Grupo.insertMany([
        { nombreGrupo: "Grupo A", cantidadAlumnos: 10
```

```

        },
        { nombreGrupo: "Grupo B", cantidadAlumnos: 15
        }
    ]);
    console.log("Grupos de ejemplo creados");
}

// Endpoints Usuarios
app.get("/usuarios", async (req, res) => {
    const usuarios = await Usuario.find();
    res.json(usuarios);
});

app.post("/crearusuario", async (req, res) => {
    try {
        const nuevoUsuario = new Usuario(req.body);
        const savedUsuario = await nuevoUsuario.save
            ();
        res.status(201).json(savedUsuario);
    } catch (error) {
        res.status(400).json({ error: error.message
        });
    }
});

app.put("/usuarios/:id", async (req, res) => {
    try {
        const usuarioActualizado = await Usuario.
            findByIdAndUpdate(
                req.params.id,
                req.body,
                { new: true }
            );
        if (!usuarioActualizado) return res.status
            (404).json({ error: "Usuario no encontrado
            " });
        res.json(usuarioActualizado);
    } catch (error) {
        res.status(400).json({ error: error.message
        });
    }
});

```

```

    }
  });

  app.delete("/usuarios/:id", async (req, res) => {
    try {
      const usuarioEliminado = await Usuario.
        findByIdAndDelete(req.params.id);
      if (!usuarioEliminado) return res.status(404)
        .json({ error: "Usuario no encontrado" });
      res.json({ mensaje: "Usuario eliminado
        correctamente" });
    } catch (error) {
      res.status(400).json({ error: error.message
        });
    }
  });

  // Endpoints Grupos
  app.get("/grupos", async (req, res) => {
    const grupos = await Grupo.find();
    res.json(grupos);
  });

  app.post("/grupos", async (req, res) => {
    try {
      const nuevoGrupo = new Grupo(req.body);
      const savedGrupo = await nuevoGrupo.save();
      res.status(201).json(savedGrupo);
    } catch (error) {
      res.status(400).json({ error: error.message
        });
    }
  });

  app.put("/grupos/:id", async (req, res) => {
    try {
      const grupoActualizado = await Grupo.
        findByIdAndUpdate(
          req.params.id,
          req.body,

```



```

        { new: true }
    );
    if (!grupoActualizado) return res.status(404)
        .json({ error: "Grupo no encontrado" });
    res.json(grupoActualizado);
} catch (error) {
    res.status(400).json({ error: error.message
        });
}
});

app.delete("/grupos/:id", async (req, res) => {
    try {
        const grupoEliminado = await Grupo.
            findByIdAndDelete(req.params.id);
        if (!grupoEliminado) return res.status(404).
            json({ error: "Grupo no encontrado" });
        res.json({ mensaje: "Grupo eliminado
            correctamente" });
    } catch (error) {
        res.status(400).json({ error: error.message
            });
    }
});

const PORT = 3000;
app.listen(PORT, () => console.log('Servidor en http
    ://localhost:${PORT}'));

} catch (error) {
    console.error("Error al conectar a MongoDB:", error);
}
}

iniciarServidor();

```

6. Endpoints de la API

[Volver al índice](#)

6.1. Usuarios

GET /usuarios

Descripción: Obtiene todos los usuarios registrados.

POST /crearusuario

Descripción: Crea un nuevo usuario.

Recibe: JSON con los datos del usuario.

PUT /usuarios/:id

Descripción: Actualiza un usuario existente.

Recibe: JSON con los campos a actualizar.

Parámetro URL: :id = ID del usuario.

DELETE /usuarios/:id

Descripción: Elimina un usuario por su ID.

Recibe: Ningún body.

7. Pruebas con Postman

[Volver al índice](#)

- Seleccionar el método HTTP.
- Introducir la URL, por ejemplo: `http://localhost:3000/crearusuario`.
- Enviar JSON en Body para POST y PUT.

7.1. GET de Usuarios

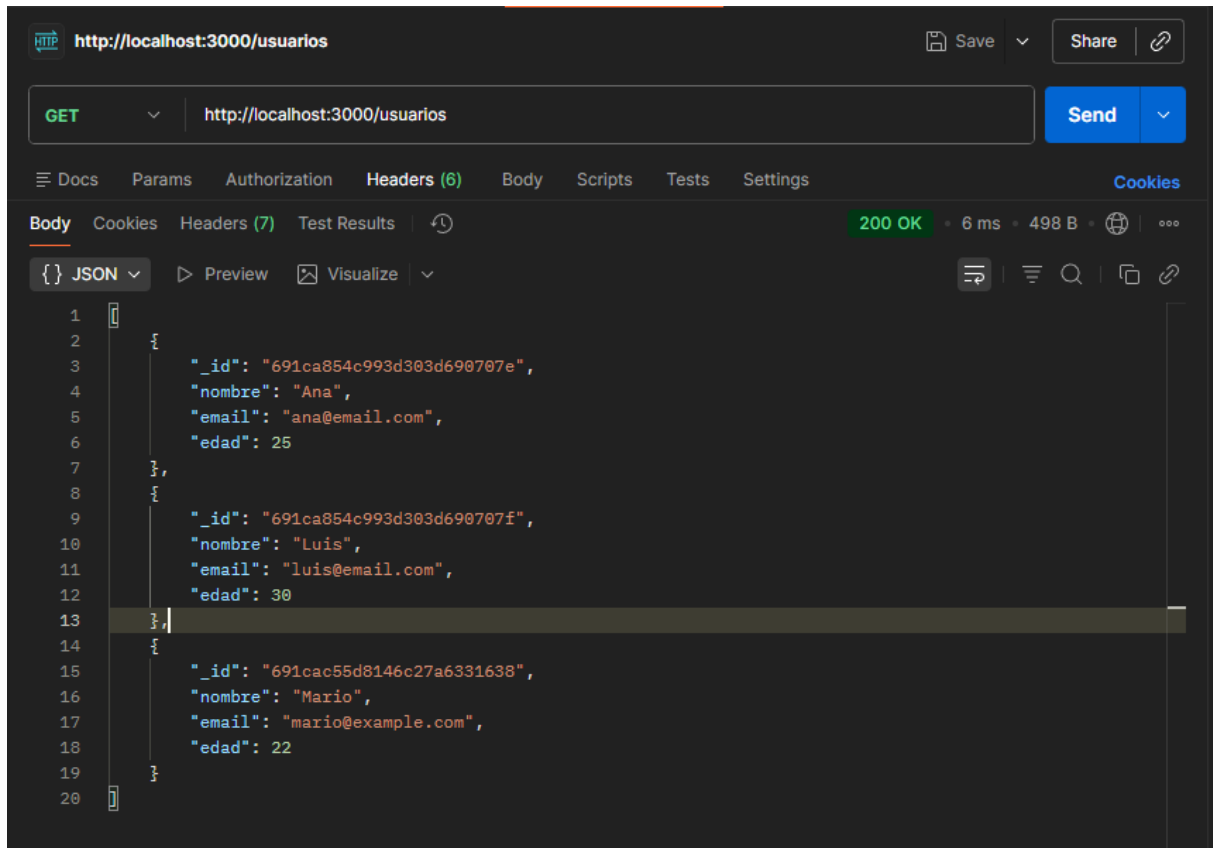


Figura 1: Visualización de los usuarios.

7.2. POST de Usuarios

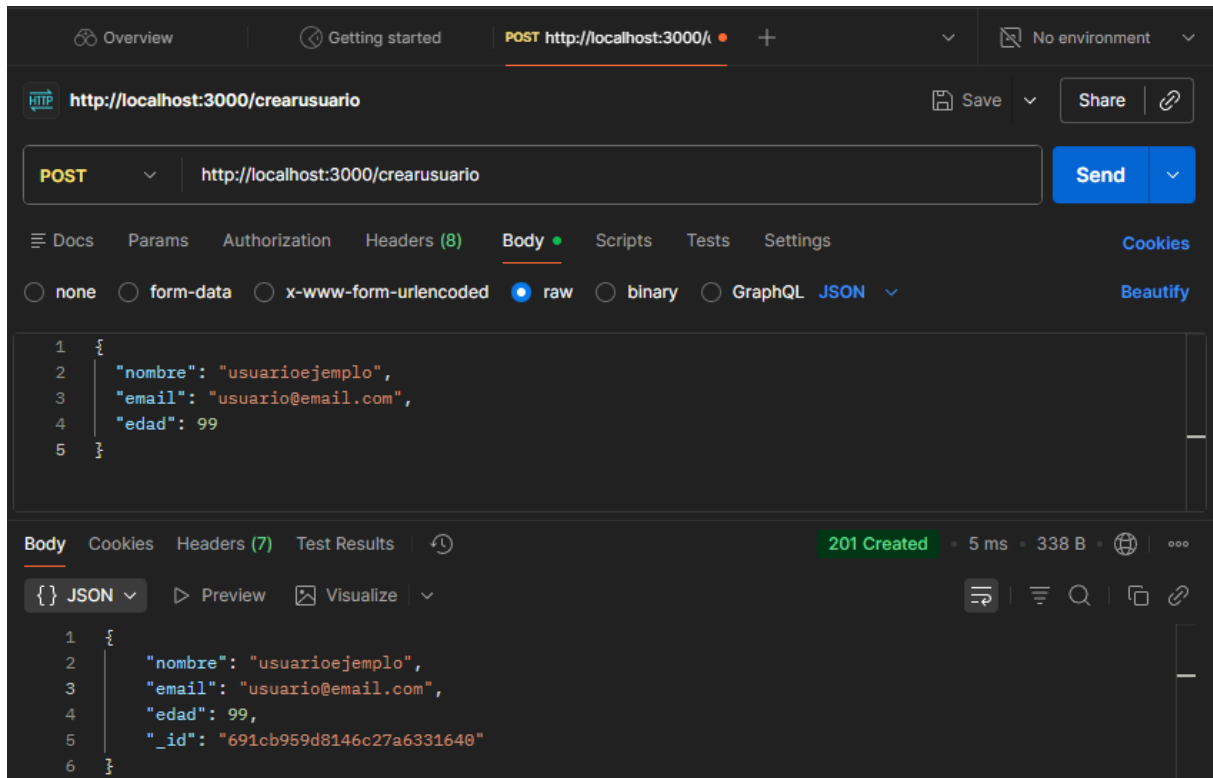


Figura 2: Creación de un nuevo usuario.

7.3. DELETE de Usuarios

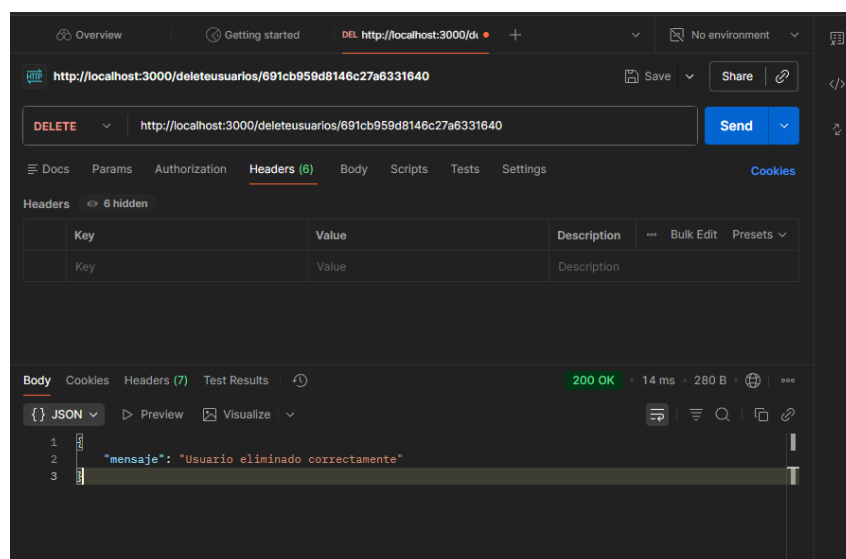


Figura 3: Eliminación de un usuario.

7.4. UPDATE de Usuarios

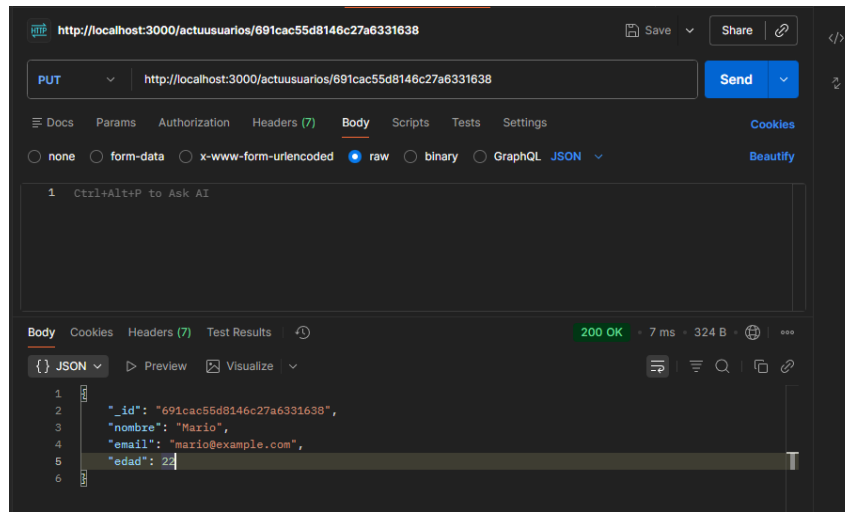


Figura 4: Actualización de usuario - paso 1.

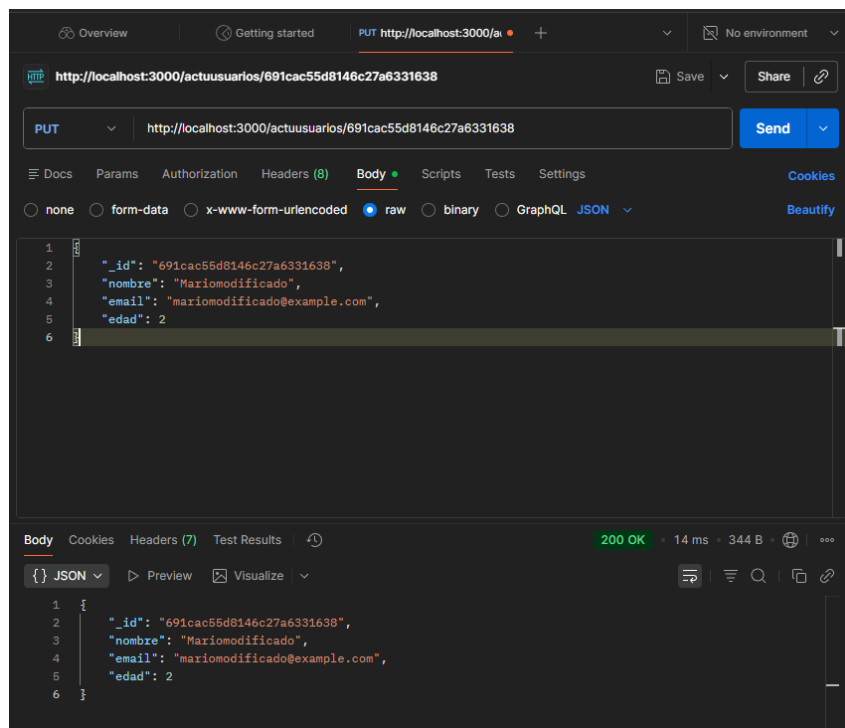


Figura 5: Actualización de usuario - paso 2.

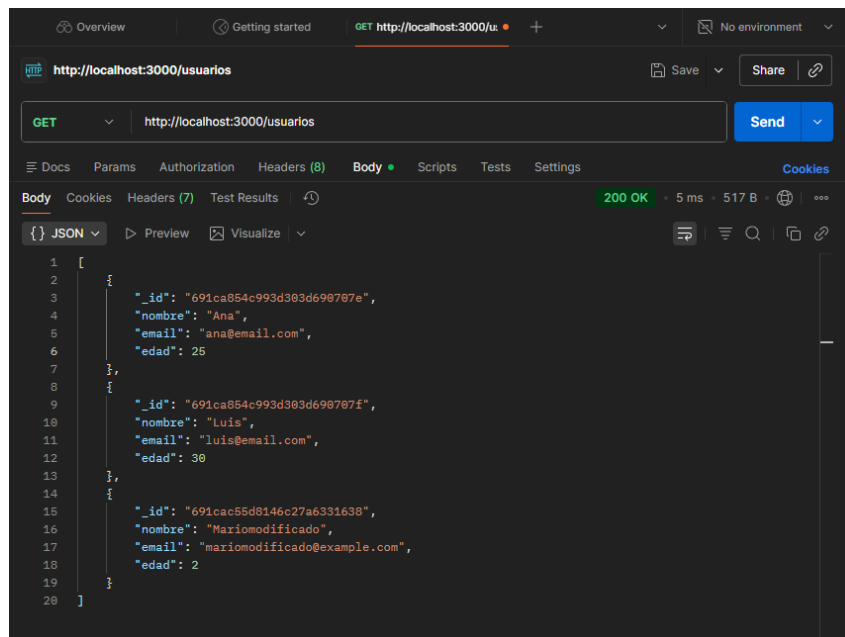


Figura 6: Actualización de usuario - paso 3

8. Docker Compose

[Volver al índice](#)

En esta sección dockerizaremos nuestra API, que originalmente corría en local, para ejecutarla dentro de un contenedor Docker.

Para ello necesitaremos dos archivos:

- `docker-compose.yml`
- `Dockerfile`

Artivo `docker.composer.yml`

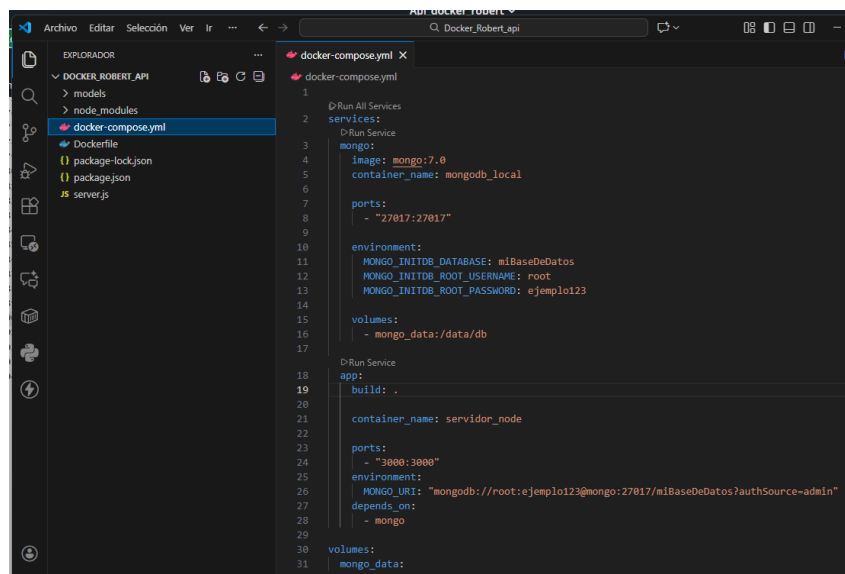


Figura 7: Esquema general de los servicios en Docker Compose.

Archivo `docker-compose.yml`

```
services:
  # Servicio de base de datos MongoDB
  mongo:
    image: mongo:7.0
    container_name: mongoddb_local
    ports:
      - "27017:27017"
    environment:
      MONGO_INITDB_DATABASE: miBaseDeDatos
      MONGO_INITDB_ROOT_USERNAME: root
      MONGO_INITDB_ROOT_PASSWORD: ejemplo123
```

```

volumes:
  - mongo_data:/data/db

# Servicio de la API Node.js
app:
  build: . # Construye la imagen usando el Dockerfile
  container_name: servidor_node
  ports:
    - "3000:3000"
  environment:
    MONGO_URI: "mongodb://root:ejemplo123@mongo:27017/
      miBaseDeDatos?authSource=admin"
  depends_on:
    - mongo

# Volumen persistente para MongoDB
volumes:
  mongo_data:

```

Archivo Dockerfile

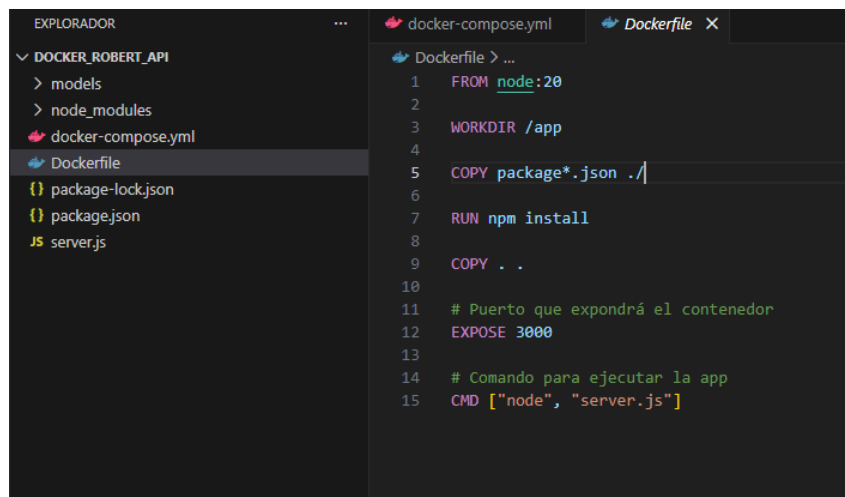


Figura 8: Dockerfile para construir la imagen de la API Node.js.

```

FROM node:20

WORKDIR /app

```



```
COPY package*.json ./

RUN npm install

COPY . .

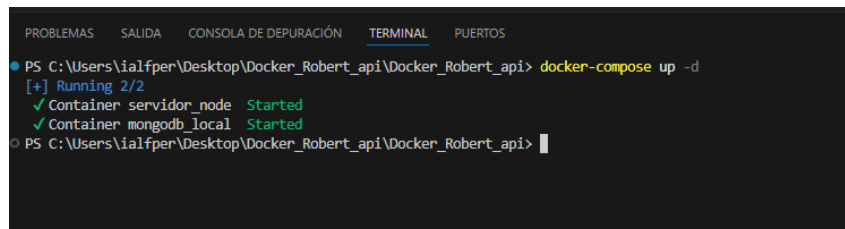
# Puerto que expondr el contenedor
EXPOSE 3000

# Comando para ejecutar la app
CMD ["node", "server.js"]
```

Levantar los contenedores

Para levantar los contenedores (MongoDB + API) ejecutamos:

```
ocker compose up -d
```



```
PROBLEMAS  SALIDA  CONSOLA DE DEPURACIÓN  TERMINAL  PUERTOS
● PS C:\Users\ialfper\Desktop\Docker_Robert_api\Docker_Robert_api> docker-compose up -d
[+] Running 2/2
  ✓ Container servidor_node Started
  ✓ Container mongodb_local Started
○ PS C:\Users\ialfper\Desktop\Docker_Robert_api\Docker_Robert_api> 
```

Figura 9: Terminal ejecutando Docker Compose.

9. Creación de la imagen de la API

Creamos una imagen propia de la API con:

```
docker build -t tuusuarioDocker/nombreDeLaImagen .
```

Luego modificamos el archivo `docker-compose.yml` para usar la imagen creada:

```
pp:
  image: ejemplo-de-una-api:v1.0.0
  container_name: miMegaApi
```

9.1. Subir la imagen a Docker Hub

1. Listar imágenes locales:

```
docker images
```

2. Subir la imagen a Docker Hub:

```
docker push miusuario/miapi:1.0
```

3. Verificar en Docker Hub → Repositories que la imagen se subió correctamente.

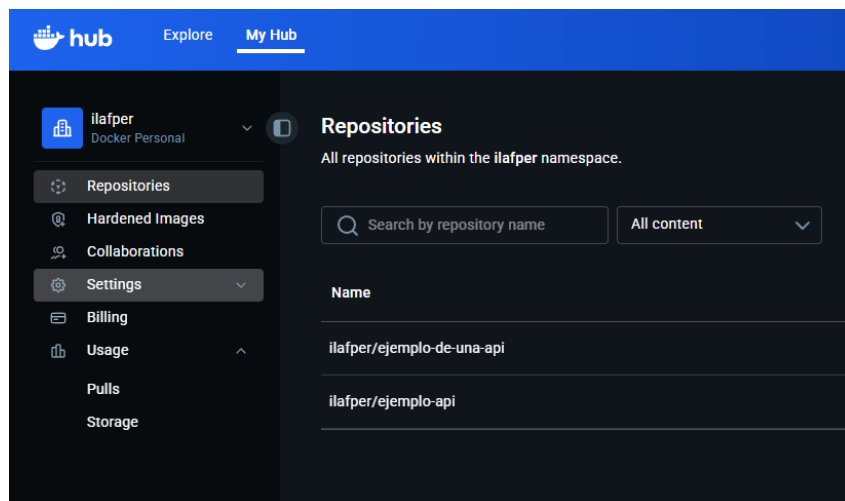


Figura 10: Imagen de la API subida a Docker Hub.

10. Workflow de GitHub

[Volver al índice](#)

¿Qué es un Workflow?

Un Workflow es un conjunto de acciones automáticas que GitHub ejecuta cuando ocurre un evento en tu repositorio, como un push, pull request o release.

En nuestro caso, el Workflow construirá y subirá automáticamente la imagen de la API a Docker Hub cada vez que hagamos push, evitando levantar manualmente el contenedor.

10.1. Creación del Workflow

Creamos la carpeta dentro del proyecto:

```
.github/workflows/
```

y dentro de esa carpeta el archivo:

```
docker-push.yml
```

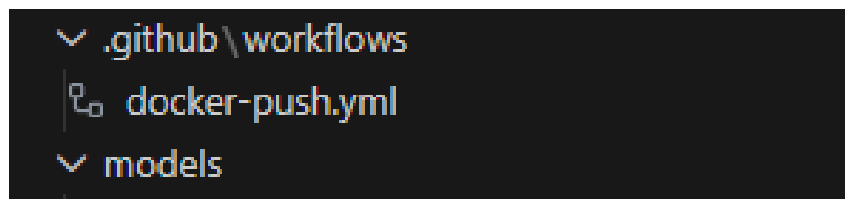


Figura 11: Carpeta y archivo Workflow en el repositorio.

Contenido del archivo docker-push.yml

```
name: Build and Push Docker Image

on:
  push:
    branches:
      - main
      - master
    paths:
      - '**'
      - '.github/workflows/docker-push.yml'
  workflow_dispatch:
```

```

env:
  DOCKER_IMAGE_NAME: ${ secrets.DOCKER_USERNAME }}/ejemplo-
    de-una-api
  DOCKER_TAG: latest

jobs:
  build-and-push:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout c digo
        uses: actions/checkout@v4

      - name: Configurar Docker Buildx
        uses: docker/setup-buildx-action@v3

      - name: Login a Docker Hub
        uses: docker/login-action@v3
        with:
          username: ${ secrets.DOCKER_USERNAME }
          password: ${ secrets.DOCKER_PASSWORD }

      - name: Construir y subir imagen Docker
        uses: docker/build-push-action@v5
        with:
          context: ./
          file: ./Dockerfile
          push: true
          tags: ${ env.DOCKER_IMAGE_NAME }:${ env.
            DOCKER_TAG }
          cache-from: type=registry,ref=${ env.
            DOCKER_IMAGE_NAME }:buildcache
          cache-to: type=inline

      - name: Mostrar informaci n de la imagen
        run: |
          echo "Imagen construida y subida exitosamente:"
          echo "  - Imagen: ${ env.DOCKER_IMAGE_NAME }:${ env.
            DOCKER_TAG }"
          echo "  - Docker Hub: https://hub.docker.com/r/${ secrets.DOCKER_USERNAME }/ejemplo-de-una-api"

```

Creación del token en Docker Hub

- Ir a Docker Hub → Settings → Personal → Generate new token - Dar un nombre al token, seleccionar permisos (lectura/escritura/borrado) y duración (puede ser permanente) - Copiar el token (una vez generado no se podrá volver a ver)

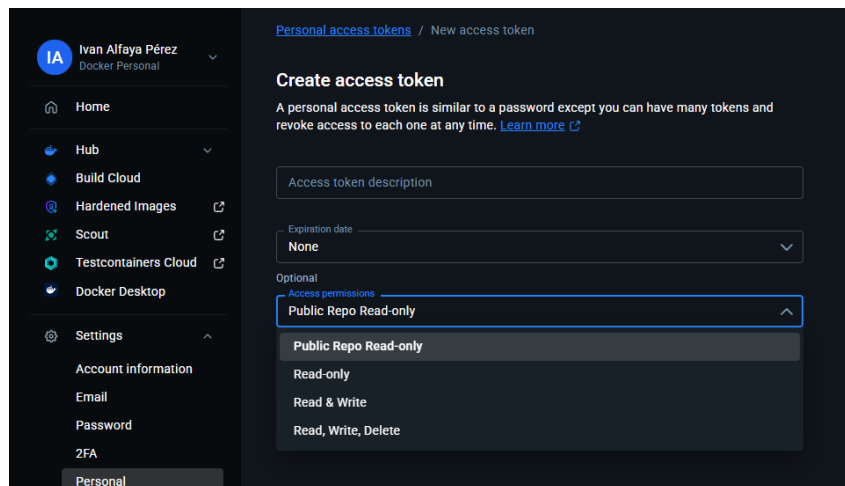


Figura 12: Token generado en Docker Hub.

- En GitHub → Settings → Secrets and variables → Actions crear: - `DOCKER_USERNAME` usuario Docker
`DOCKER_PASSWORD` token generado

Resultado

- Hacer push al repositorio ejecuta automáticamente el Workflow - La imagen Docker de la API se construye y se sube a Docker Hub - Ya no es necesario levantar manualmente el contenedor para tener la versión más reciente

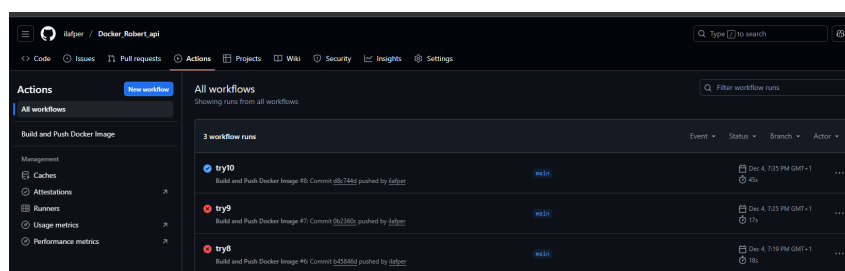


Figura 13: Workflow ejecutado en GitHub Actions.